

```

---
title: "Ejercicio_Modulo2"
author: "KPI Rojos"
date: "2026-04-15"
output: slidy_presentation
---
## <span style="color: red;"><u>La penitencia de Newton:</u></span>
Desarrolla dos algoritmos que hagan este trabajo, por ejemplo para sumar desde 1 hasta 1 * 1010 y verifica cuál de los dos es más eficiente

![[Imagen de Newton](https://encrypted-tbn0.gstatic.com/licensed-image?q=tbn:AND9GcRpGcvvt8nF0dnaBW0XtwU82bZRus4R1AFNnyHlpwDxyWGswXYq691VH9t8Ku16Ip0Ig55Iq3BCy6vtJ1VeSeITf7qH0HNJ9KoNtVQJs2RuSdFqBY1vj2TYo0cKmvfoiVGzCjR5

## <span style="color: red;"><u>Algoritmo 1:</u></span>
El primer método que decidimos hacer es el iterativo, se basa en carear un vector de longitud del número elegido. Luego de eso se procede a
sumar ciclicamente un número luego el siguiente y así sucesivamente.
Este método tiene como desventaja clara un mayor tiempo de demora, si bien en números relativamente chicos para la potencia de las computadoras
(como lo es 1010) no hay tanta diferencia, en números mas grandes si es notoria la diferencia.

```{r}
Newton <- (1:1010)
library(tictoc)

Simulamos Newton (asegúrate de que Newton sea un vector numérico)
Newton <- c(...)

tic("Tiempo algoritmo 1 Newton:")

total <- 0
valor_final <- length(Newton)

for (i in 1:valor_final) {
 total <- total + i
total
}
toc()
print(total)

```
<span style="color: green;">Aclaración, tuvimos que instalar previamente la libreria tictoc para que el sistema pueda contar el tiempo que se
demoro el programa. La instalación de las librerias es mediante: "tools"->"Instal Packages" y elegir la libreria deseada.</span>

## <span style="color: red;"><u>Algoritmo 2:</u></span>
El segundo método se basa en usar la formula de la progresión aritmetica tambien conocida como la fórmula de Gauss.
La ventaja de este metodo es que no es necesario crear un vector ni ningun otro problema, simplemente con una fórmula matemática.

```{r}
library(tictoc)

tic("Tiempo algoritmo 2 Newton:")
inicio<- 1
final <-1010
newton2 <- (final*(final+1))/2
toc()
print (newton2)

```
## <span style="color: red;"><u>Aplicación general:</u></span>
Ademas este metodo con este método se puede generalizar aplicando la fórmula general
```{r}
inicio<- 1
final <-1010
n<-final-inicio+1
newton2 <- (n*(final+inicio))/2
print (newton2)
```
Donde:

- Inicio es el primer número.

- Final es el último número.

- n es la cantidad total de términos que estás sumando.

## <span style="color: red;"><u>Ventajas y desventajas:</u></span>

<span style="font-size: 20px;">
<table border="1">
<tr>
  <span style="color: #c0392b;"><th> Metodo </th>
  <th> Ventaja </th>
  <th> Desventaja </th></span>
</tr>

<tr>
  <span style="color: blue;"><th> Iterativo </th> </span>
  <td> Mas intuitivo </td>
  <td> Programacion mas extensa y mas lento </td>
</tr>

<tr>
  <span style="color: blue;"><th> Formula </th> </span>
  <td> Mas rapido y facil de programar </td>
  <td> Conocimiento previo en caso de no poder buscar la formula </td>
</tr>
</table>
</span>
<span style="font-size: 14px;">
<span style="color: green;"> La lista, tabla y los colores y tamaño de las fuentes se programo mediante etiquetas de html, es una de lasss
posibilidades que permite la programacion en Rmd con formato html</span></span>

```